# CMPSC 465 Homework 6 Solutions

Mohammad Abu Shariha

Fall 2025

## Problem 1(a): Dijkstra with Reinsertion – Correctness

This variant of Dijkstra's algorithm allows a vertex to be reinserted into the priority queue if a later update reduces its distance. Unlike the standard Dijkstra's algorithm, this version **does correctly compute the shortest-path distances** even when some edges have negative weights, provided there are **no negative-weight cycles**.

**Reasoning:**

- Each time an edge $(u, v)$ is relaxed, $d[v]$ is updated to the length of some concrete path from $s$ to $v$.
- Whenever a smaller distance is found, $v$ is reinserted into the queue so the improvement can propagate to its neighbors.
- Because there are no negative cycles, every vertex's distance value can only decrease a finite number of times (each decrease corresponds to a strictly shorter simple path).
- When the priority queue becomes empty, no edge $(u, v)$ can further reduce any $d[v]$, meaning the distances satisfy the Bellman optimality conditions.

**Tightened proof sketch:** Every time a relaxation decreases dist$[v]$ it sets it to the length of some concrete path from $s$ to $v$, so stored distances are always realizable path lengths. Because there is no negative-weight cycle, any simple path can only strictly decrease a finite number of times, so repeated strict decreases must terminate. When the priority queue becomes empty no relaxation can further decrease any dist$[\cdot]$, hence the stored distances satisfy the Bellman optimality inequalities $\text{dist}[v] \leq \text{dist}[u] + \ell(u, v)$ for every edge $(u, v)$. These inequalities characterize shortest-path distances, so the final distances are shortest paths from $s$. This is the same termination+Bellman-inequalities argument used for label-correcting algorithms (e.g., Bellman–Ford).

Thus, all vertices have their true shortest-path distances when the algorithm terminates. The reinsertion turns Dijkstra into a label-correcting algorithm that behaves like Bellman–Ford with a priority queue, guaranteeing correctness.

**Conclusion:** The algorithm is correct for graphs with negative edges but no negative cycles, although potentially inefficient.

## Problem 1(b): Dijkstra with Reinsertion – Runtime Complexity

Let $V = |V|$ and $E = |E|$. Each relaxation may trigger an insertion into the priority queue, and each insertion or extraction costs $O(\log V)$. In the worst case the total number of edge relaxations performed by a label-correcting scheme of this kind is $O(VE)$ (matching the Bellman–Ford bound).

$$\text{Runtime: } O(VE \log V)$$

In pathological examples the algorithm can perform up to $O(VE)$ relaxations; multiplying by the $O(\log V)$ heap operation cost yields the stated bound. This matches Bellman–Ford's $O(VE)$ bound up to a $\log V$ factor. In practice it behaves more efficiently when few reinsertions occur.

## Problem 2: Budget Bellman-Ford

We are asked to find the shortest path from $s$ to each vertex $v$ using at most $k$ edges.

### Algorithm

We use dynamic programming similar to Bellman–Ford:

Let $d[i][v]$ denote the minimum distance from $s$ to $v$ using at most $i$ edges.

$$\begin{cases} d[0][s] = 0, \\ d[0][v] = \infty, \quad v \neq s \end{cases}$$

For $i = 1, 2, \ldots, k$:

$$d[i][v] = \min\left(d[i-1][v], \min_{(u,v)\in E}(d[i-1][u] + \ell(u,v))\right)$$

The final answer for each vertex $v$ is $d[k][v]$ (shortest path using *at most* $k$ edges).

## Problem 2(b): Correctness and Complexity

**Correctness:** By induction on $i$, $d[i][v]$ represents the minimum distance using at most $i$ edges. The recurrence considers either staying at $v$ or reaching it through one additional edge from a previously optimal path of length $i-1$. Thus, correctness follows.

**Complexity:** Each iteration processes all edges once.

$$\text{Time: } O(kE), \quad \text{Space: } O(kV)$$

The naive DP stores all $k+1$ layers (space $O(kV)$), but this can be reduced to $O(V)$ by using a rolling array that keeps only the previous layer when computing the next. If $k \geq V-1$ the algorithm becomes equivalent to Bellman–Ford (it computes unconstrained shortest paths).

## Problem 3(a): Ordered Bellman-Ford – Correctness

Ordering edges by nondecreasing weight before each round does not affect correctness. Bellman–Ford remains correct as long as each edge is relaxed enough times to enforce all shortest-path inequalities. The order only changes convergence speed, not final values, since every edge is still relaxed in each round.

**Formal remark:** Ordering the edges within a round cannot break correctness because each full round still relaxes every edge once; correctness follows from the same Bellman-inequality argument as in standard Bellman–Ford.

## Problem 3(b): Ordered Bellman-Ford – Fewer Rounds Example

Consider:

$$s \xrightarrow{1} a \xrightarrow{1} b, \quad s \xrightarrow{100} b$$

If we process edges in arbitrary order, we might first set $d[b] = 100$, and only later improve it to 2. However, with edges sorted by weight, $(s, a)$ and $(a, b)$ are processed first, giving $d[b] = 2$ immediately. Thus, fewer rounds are needed.

## Problem 3(c): Ordered Bellman-Ford – No Improvement Example

For a linear chain $s \to a \to b \to c$ with all edge weights $= 1$, each iteration can only propagate the shortest path one vertex farther, regardless of order. Hence, $|V| - 1$ rounds are still required.

## Problem 4(a): Pinned Nodes – Algorithm

We need $d(i, j)$, the shortest path from $i$ to $j$ that includes at least one vertex from a special subset $S \subseteq V$.

Use Floyd–Warshall to precompute all-pairs shortest paths $d(i, j)$.

Then compute:

$$d_S(i, j) = \min_{k \in S}[d(i, k) + d(k, j)]$$

If $i \in S$ or $j \in S$, they are automatically valid paths because the requirement is "includes at least one vertex from $S$." If no such $k$ yields a finite value, set $d_S(i, j) = \infty$.

## Problem 4(b): Pinned Nodes – Correctness and Complexity

Every path from $i$ to $j$ passing through some $k \in S$ can be decomposed into subpaths $i \to k$ and $k \to j$. Taking the minimum over all $k$ ensures the path includes at least one pinned node.

**Remark:** Floyd–Warshall correctly computes $d(i, j)$ even when some edge weights are negative (so long as there are no negative cycles), so the values $d(i, k) + d(k, j)$ used above are valid. The extra minimization over $k \in S$ costs $O(|S|n^2)$ time, which does not change the $O(n^3)$ asymptotic bound.

**Complexity:**

$O(n^3)$ time and $O(n^2)$ space.

## Problem 5(a): Bottleneck Path – Algorithm

Given graph $G = (V, E)$ with edge weights $w(u, v) \geq 0$, define the bottleneck weight of a path as the maximum edge weight along it. We want:

$$b(i, j) = \min_{p: i \to j} \max_{(u,v) \in p} w(u, v)$$

**Algorithm (Floyd–Warshall variant):**

$$b[i][j] = \begin{cases} \min\{w_e \mid e \text{ is an edge } (i, j)\}, & \text{if one or more edges } (i, j) \in E, \\ \infty, & \text{otherwise.} \end{cases}$$

(If multiple parallel edges exist between $i$ and $j$, initialize $b[i][j]$ to the minimum of their weights.) Initialize $b[i][i] = 0$ for all $i$. **Note:** $b[i][i] = 0$ because the empty path from a vertex to itself has no edges, so its maximum edge weight is conventionally taken as 0.

For all $k, i, j$:

$$b[i][j] = \min(b[i][j], \max(b[i][k], b[k][j]))$$

## Problem 5(b): Bottleneck Path – Correctness and Complexity

**Correctness:** Inductively, after considering vertices $\{1, \ldots, k\}$, $b[i][j]$ stores the minimum bottleneck value for all paths whose internal vertices are in that subset. When $k = n$, we have the correct minimum bottleneck for all pairs. Since weights are nonnegative, the max and min operations are well-defined as used above.

**Complexity:**

$O(n^3)$ time, $O(n^2)$ space.